



**INSTITUTO POLITÉCNICO NACIONAL.
UNIDAD PROFESIONAL INTERDISCIPLINARIA EN
INGENIERÍA Y TECNOLOGÍAS AVANZADAS**



Unidad de aprendizaje:
Bases de datos distribuidas.

Practica 4: Problemas de conurrencia.

Alumnos:

- Fuentes García Liliana Alejandra.
- Marcelino Cerón Diego Antonio

Profesora:

Carlos De La Cruz Sosa.

Fecha de entrega:

24 de septiembre del 2026.

Lectura sucia.

La lectura sucia es un problema de concurrencia que ocurre cuando una transacción lee datos modificados por otra transacción que todavía no ha confirmado. Si la primera transacción realiza un ROLLBACK, los datos leídos por la segunda nunca se guardan en la base de datos.

A modifica sin confirmar -> B lee el valor provisional -> A revierte -> B había leído un dato que nunca quedó guardado.

En este ejemplo la transacción A modifica el nombre de una persona en la tabla Person.Person sin confirmar el cambio. Posteriormente, la transacción B utiliza el nivel de aislamiento READ UNCOMMITTED para leer el valor provisional. Finalmente, A ejecuta un ROLLBACK, deshaciendo la modificación. De esta manera, se demuestra que B pudo leer un dato que nunca se confirmó en la base de datos.

Ventana A:

```
-- VENTANA A1: deja abierta la transacción, no confirmes.
USE AdventureWorks2022;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRANSACTION;
UPDATE Person.Person
SET FirstName = N'NombreSucio'
WHERE BusinessEntityID = 1;
SELECT 'A: cambio provisional' AS Paso, BusinessEntityID, FirstName
FROM Person.Person WHERE BusinessEntityID = 1;

-- VENTANA A2: revierte la modificación provisional.
ROLLBACK TRANSACTION;
SELECT 'A: despues del ROLLBACK' AS Paso, BusinessEntityID, FirstName
FROM Person.Person WHERE BusinessEntityID = 1;
GO
```

Ventana B:

```
-- VENTANA B1: READ UNCOMMITTED permite leer lo que A no ha confirmado.
USE AdventureWorks2022;
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
SELECT 'B: lectura sucia esperada: NombreSucio' AS Paso,
      BusinessEntityID, FirstName
FROM Person.Person WHERE BusinessEntityID = 1;

-- VENTANA B2: debe mostrar el nombre original, no NombreSucio.
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT 'B: valor confirmado' AS Paso, BusinessEntityID, FirstName
FROM Person.Person WHERE BusinessEntityID = 1;
GO
```

La prueba demuestra cómo se presenta una lectura sucia como problema de concurrencia. La transacción B puede leer el valor NombreSucio mientras la transacción A todavía mantiene el cambio sin confirmar. Sin embargo, cuando A ejecuta ROLLBACK, ese valor desaparece porque nunca llegó a guardarse de manera permanente en la base de datos. Por lo tanto, B terminó leyendo información que realmente nunca fue confirmada. Para evitar este problema, se pueden utilizar niveles de aislamiento como READ COMMITTED o SERIALIZABLE. En este ejercicio, para poder reproducir intencionalmente la anomalía se utiliza READ UNCOMMITTED en la transacción B. También sería posible utilizar el hint NOLOCK, que permite realizar lecturas sin esperar los bloqueos y puede producir este mismo tipo de lectura sucia.

○ **Resultados ventana A1**

	Paso	BusinessEntityID	FirstName
1	A: cambio provisional	1	NombreSucio

○ **Resultados ventana B1:**

	Paso	BusinessEntityID	FirstName
1	B: lectura sucia esperada: NombreSucio	1	NombreSucio

○ **Resultado ventana A2:**

	Paso	BusinessEntityID	FirstName
1	A: despues del ROLLBACK	1	Ken

○ **Resultados ventana B2:**

	Paso	BusinessEntityID	FirstName
1	B: valor confirmado	1	Ken

Lectura no repetible.

La lectura no repetible es un problema de concurrencia que ocurre cuando una transacción lee un dato y, al volver a leerlo dentro de la misma transacción, obtiene un valor diferente porque otra transacción lo modificó y confirmó entre ambas lecturas.

A cuenta los pedidos -> B cambia la cantidad -> A calcula el total -> B cambia el descuento -> A compara un promedio calculado con el total anterior contra uno actualizado.

El ejemplo demuestra una lectura no repetible utilizando dos ventanas de SQL Server. En la ventana A se realiza una primera lectura del nombre de una persona y se mantiene abierta

la transacción. Después, la ventana B modifica y confirma el nombre. Cuando A vuelve a consultar el mismo registro, obtiene un valor diferente al de la primera lectura.

Ventana A:

```
--Ventana A – A1: cuenta los pedidos con detalles y deja abierta la transacción.

USE AdventureWorks2022;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

IF OBJECT_ID('tempdb..#AnálisisNoRepetible') IS NOT NULL
    DROP TABLE #AnálisisNoRepetible;

CREATE TABLE #AnálisisNoRepetible
(
    CantidadPedidos int NOT NULL,
    TotalEnA2 decimal(38,4) NULL
);

BEGIN TRANSACTION;

INSERT INTO #AnálisisNoRepetible (CantidadPedidos)
SELECT COUNT(DISTINCT SalesOrderID)
FROM Sales.SalesOrderDetail;

SELECT 'A1: pedidos con al menos un detalle' AS Paso,
    CantidadPedidos
FROM #AnálisisNoRepetible;

--Ventana A – A2: calcula el total después del cambio de cantidad hecho por B.
UPDATE #AnálisisNoRepetible
SET TotalEnA2 =
(
    SELECT SUM
    (
        CONVERT(decimal(19,4),
            UnitPrice * OrderQty * (1.0 - UnitPriceDiscount))
    )
    FROM Sales.SalesOrderDetail
);

SELECT 'A2: total calculado después del cambio de cantidad' AS Paso,
    TotalEnA2
FROM #AnálisisNoRepetible;

--Ventana A – A3: termina la transacción y compara los promedios.
SELECT
    'Promedio con el total que A guardó en A2' AS Calculo,
    TotalEnA2 / NULLIF(CantidadPedidos, 0) AS Promedio
FROM #AnálisisNoRepetible

UNION ALL

SELECT
    'Promedio recalculado con los datos actuales',
    AVG(TotalPorPedido)
FROM
(
```

```

SELECT
    SUM(CONVERT(decimal(19,4),
        UnitPrice * OrderQty * (1.0 - UnitPriceDiscount)))
    AS TotalPorPedido
FROM Sales.SalesOrderDetail
GROUP BY SalesOrderID
) AS TotalesActuales;

COMMIT TRANSACTION;
GO

```

Ventana B:

```

--Ventana B – B1: cambia la cantidad del detalle seleccionado.
--Al ser una instrucción sin transacción explícita, se confirma automáticamente.

USE AdventureWorks2022;
GO

UPDATE D
SET OrderQty =
    CASE WHEN D.OrderQty > 1 THEN D.OrderQty - 1 ELSE D.OrderQty + 1 END
FROM Sales.SalesOrderDetail AS D
JOIN dbo.ConcurrencyRestoreDetail AS R
    ON R.SalesOrderID = D.SalesOrderID
   AND R.SalesOrderDetailID = D.SalesOrderDetailID;

SELECT 'B1: cantidad modificada y confirmada' AS Paso,
    D.SalesOrderID, D.SalesOrderDetailID, D.OrderQty
FROM Sales.SalesOrderDetail AS D
JOIN dbo.ConcurrencyRestoreDetail AS R
    ON R.SalesOrderID = D.SalesOrderID
   AND R.SalesOrderDetailID = D.SalesOrderDetailID;

--Ventana B – B2: cambia y confirma el descuento del mismo detalle.
USE AdventureWorks2022;

UPDATE D
SET UnitPriceDiscount =
    CASE WHEN D.UnitPriceDiscount = 0.50 THEN 0.00 ELSE 0.50 END
FROM Sales.SalesOrderDetail AS D
JOIN dbo.ConcurrencyRestoreDetail AS R
    ON R.SalesOrderID = D.SalesOrderID
   AND R.SalesOrderDetailID = D.SalesOrderDetailID;

SELECT 'B2: descuento modificado y confirmado' AS Paso,
    D.SalesOrderID, D.SalesOrderDetailID, D.UnitPriceDiscount
FROM Sales.SalesOrderDetail AS D
JOIN dbo.ConcurrencyRestoreDetail AS R
    ON R.SalesOrderID = D.SalesOrderID
   AND R.SalesOrderDetailID = D.SalesOrderDetailID;

```

Al comparar los dos promedios, se observa una diferencia debido a que el segundo cálculo se realiza después del cambio de descuento efectuado por B. Esto muestra que A obtiene

información diferente en cada consulta, aunque ambas se realizan dentro de la misma transacción. Con READ COMMITTED, cada consulta utiliza datos confirmados, pero los bloqueos no se mantienen entre consultas, permitiendo que otra transacción modifique los datos y cambie el resultado. Además, READ COMMITTED libera los bloqueos de lectura al terminar cada SELECT, por lo que B pudo actualizar y confirmar entre ambas lecturas. REPEATABLE READ mantiene protegida la fila leída hasta el final de la transacción; en ese nivel B esperaría y A volvería a leer el nombre original.

○ **Resultados ventana A1:**

	Paso	CantidadPedidos
1	A1: pedidos con al menos un detalle	31465

○ **Resultados ventana B1:**

	Paso	SalesOrderID	SalesOrderDetailID	OrderQty
1	B1: cantidad modificada y confirmada	43659	1	2

○ **Resultados ventana A2:**

	Paso	TotalEnA2
1	A2: total calculado después del cambio de cantidad	109848406.4190

○ **Resultados ventana B2:**

	Paso	SalesOrderID	SalesOrderDetailID	UnitPriceDiscount
1	B2: descuento modificado y confirmado	43659	1	0.50

○ **Resultados ventana A3:**

	Calculo	Promedio
1	Promedio con el total que A guardó en A2	3491.130030
2	Promedio recalculado con los datos actuales	3491.065673

Lectura Fantasma.

Una lectura fantasma es un problema de concurrencia que ocurre cuando una transacción realiza una consulta y, al repetirla, aparecen o desaparecen filas nuevas debido a que otra transacción insertó o eliminó registros entre ambas consultas.

B inserta otro detalle del mismo producto -> cuando A repite la consulta, aparece una fila adicional.

El ejemplo demuestra una lectura fantasma mediante dos transacciones. La ventana A consulta y cuenta los detalles de un producto. Después, la ventana B inserta y confirma un nuevo detalle para ese mismo producto. Cuando A repite la consulta, aparece el nuevo registro y aumenta el número de resultados, mostrando cómo los datos pueden cambiar entre dos consultas dentro de una misma transacción.

Ventana A:

```
--Ventana A - A1: consulta los detalles existentes y deja abierta la transacción.
USE AdventureWorks2022;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

BEGIN TRANSACTION;

SELECT 'A1: detalles iniciales del producto' AS Paso,
      D.SalesOrderID, D.SalesOrderDetailID, D.ProductID,
      D.OrderQty, D.UnitPrice
FROM Sales.SalesOrderDetail AS D
JOIN dbo.ConcurrencyPhantomTarget AS T
  ON T.ProductID = D.ProductID
ORDER BY D.SalesOrderID, D.SalesOrderDetailID;

SELECT 'A1: cantidad inicial de detalles' AS Paso, COUNT(*) AS Cantidad
FROM Sales.SalesOrderDetail AS D
JOIN dbo.ConcurrencyPhantomTarget AS T
  ON T.ProductID = D.ProductID;

--Ventana A - A2: repite la consulta. Debe aparecer el detalle marcado PHANTOM_DEMO
--y el conteo debe aumentar en uno.

SELECT 'A2: detalles después de la inserción' AS Paso,
      D.SalesOrderID, D.SalesOrderDetailID, D.ProductID,
      D.OrderQty, D.UnitPrice, D.CarrierTrackingNumber
FROM Sales.SalesOrderDetail AS D
JOIN dbo.ConcurrencyPhantomTarget AS T
  ON T.ProductID = D.ProductID
ORDER BY D.SalesOrderID, D.SalesOrderDetailID;

SELECT 'A2: cantidad final de detalles' AS Paso, COUNT(*) AS Cantidad
FROM Sales.SalesOrderDetail AS D
JOIN dbo.ConcurrencyPhantomTarget AS T
  ON T.ProductID = D.ProductID;

-- VENTANA A - A3: terminar la transacción después de observar la fila fantasma.
COMMIT TRANSACTION;
```

Ventana B:

```
--Ventana B - B1: inserta un detalle válido copiando los datos de otro detalle del mismo producto.
USE AdventureWorks2022;

INSERT INTO Sales.SalesOrderDetail
(SalesOrderID, CarrierTrackingNumber, OrderQty, ProductID,
SpecialOfferID, UnitPrice, UnitPriceDiscount, ModifiedDate)
SELECT TOP (1)
      D.SalesOrderID, N'PHANTOM_DEMO', D.OrderQty, D.ProductID,
      D.SpecialOfferID, D.UnitPrice, D.UnitPriceDiscount, GETDATE()
FROM Sales.SalesOrderDetail AS D
JOIN dbo.ConcurrencyPhantomTarget AS T
  ON T.ProductID = D.ProductID
ORDER BY D.SalesOrderID, D.SalesOrderDetailID;
```

```

SELECT 'B1: detalle nuevo confirmado' AS Paso,
       SalesOrderID, SalesOrderDetailID, ProductID, CarrierTrackingNumber
FROM Sales.SalesOrderDetail
WHERE CarrierTrackingNumber = N'PHANTOM_DEMO';

```

En el ejemplo se observa una lectura fantasma, ya que el registro insertado por B cumple con el filtro de A, pero no existía cuando A realizó la primera consulta. Por ello, al repetir la consulta, A obtiene un registro adicional y el conteo aumenta. Con READ COMMITTED, estos cambios pueden aparecer entre consultas. En cambio, con SERIALIZABLE, SQL Server protege el rango consultado, por lo que el INSERT de B tendría que esperar hasta que A termine su transacción. READ COMMITTED no conserva bloqueos del rango entre instrucciones. REPEATABLE READ protege filas existentes, pero no impide insertar nuevas filas que satisfagan el filtro. SERIALIZABLE protege el rango; por tanto, el INSERT concurrente de B espera hasta que A confirme.

○ Resultado ventana A1:

	Paso	SalesOrderID	SalesOrderDetailID	ProductID	OrderQty	UnitPrice
1	A1: detalles iniciales del producto	51081	35594	870	4	2,994
2	A1: detalles iniciales del producto	51083	35614	870	4	2,994
3	A1: detalles iniciales del producto	51085	35679	870	2	2,994
4	A1: detalles iniciales del producto	51090	35786	870	9	2,994
5	A1: detalles iniciales del producto	51091	35831	870	4	2,994
6	A1: detalles iniciales del producto	51092	35847	870	16	2,7445
7	A1: detalles iniciales del producto	51101	35990	870	3	2,994
8	A1: detalles iniciales del producto	51104	36078	870	4	2,994
9	A1: detalles iniciales del producto	51108	36208	870	4	2,994
10	A1: detalles iniciales del producto	51109	36261	870	18	2,7445
11	A1: detalles iniciales del producto	51110	36305	870	5	2,994
	Paso	Cantidad				
1	A1: cantidad inicial de detalles	4688				

○ Resultado ventana B1:

	Paso	SalesOrderID	SalesOrderDetailID	ProductID	CarrierTrackingNumber
1	B1: detalle nuevo confirmado	51081	121319	870	PHANTOM_DEMO

○ Resultado ventana A2:

	Paso	SalesOrderID	SalesOrderDetailID	ProductID	OrderQty	UnitPrice	CarrierTrackingNumber
1	A2: detalles después de la inserción	51081	35594	870	4	2,994	EDBF-4FE1-8F
2	A2: detalles después de la inserción	51081	121319	870	4	2,994	PHANTOM_DEMO
3	A2: detalles después de la inserción	51081	121320	870	4	2,994	PHANTOM_DEMO
4	A2: detalles después de la inserción	51083	35614	870	4	2,994	742D-4BE9-A5
5	A2: detalles después de la inserción	51085	35679	870	2	2,994	EB9F-4A76-97
6	A2: detalles después de la inserción	51090	35786	870	9	2,994	EBDE-4A71-8E
7	A2: detalles después de la inserción	51091	35831	870	4	2,994	DEAC-431D-94
8	A2: detalles después de la inserción	51092	35847	870	16	2,7445	FB68-47B7-9D
	Paso	Cantidad					
1	A2: cantidad final de detalles	4690					